

HerAI Fellowship

BACK-END DEVELOPMENT FOR AI PRODUCTS

Modul Belajar Praktis dan Mudah Dipahami

Membangun layanan belakang AI yang terstruktur, aman, scalable, dan mudah diuji.

Level	Intermediate
Estimasi belajar	20-24 jam
Prasyarat	Python, database dasar, dan HTTP dasar
Versi	1.0 - 23 Juli 2026
Route	/participant/courses/back-end-development-for-ai

Disusun untuk Participant Module - Project HerAI

Tentang Modul

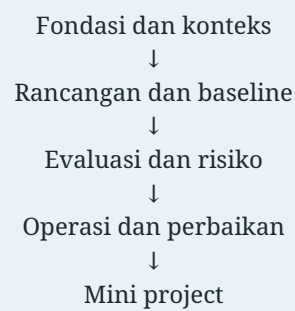
Back-end Development for AI Products membahas API, data model, authentication, service layer, asynchronous jobs, integrasi model dan RAG, caching, testing, security, serta migration.

Pola belajar: pahami tujuan, buat baseline kecil, uji kegagalan, dokumentasikan bukti, lalu perbaiki secara bertahap. Jangan menambah kompleksitas sebelum masalah dan metriknya jelas.

Cara Menggunakan Modul

- Baca tujuan dan gambaran sederhana setiap bab.
- Kerjakan checkpoint tanpa melihat kembali materi.
- Adaptasi contoh pada data aman atau sintetis.
- Catat asumsi, error, keputusan, dan keterbatasan.
- Gunakan mini project sebagai artefak portofolio.

Peta Belajar



Daftar Isi

- 01 Bab 1 - Arsitektur Back-end dan Domain**
- 02 Bab 2 - HTTP, REST, dan API Design
- 03 Bab 3 - Data Modeling dan Database
- 04 Bab 4 - Authentication dan Authorization
- 05 Bab 5 - Service Layer dan Dependency Management
- 06 Bab 6 - Asynchronous Job dan Message Queue
- 07 Bab 7 - Integrasi Model, RAG, dan Tool
- 08 Bab 8 - Caching, Rate Limit, dan Performa
- 09 Bab 9 - Testing, Security, dan Observability
- 10 Bab 10 - Deployment, Migration, dan Operasi
- 11 Bab 11 - Mini Project: Back-end Knowledge Assistant**
- 12 Bab 12 - Kuis Akhir**
- 13 Bab 13 - Diskusi dan Refleksi
- 14 Bab 14 - Glosarium dan Checklist
- 15 Bab 15 - Ringkasan dan Referensi**

HerAI Fellowship - Participant Module Membangun layanan belakang AI yang terstruktur, aman, scalable, dan mudah diuji.

Tentang Modul

Back-end Development for AI Products membahas API, data model, authentication, service layer, asynchronous jobs, integrasi model dan RAG, caching, testing, security, serta migration.

Modul disusun dengan bahasa bertahap. Setiap bab berisi tujuan, konsep inti, alur kerja, contoh kasus, kesalahan umum, checkpoint, dan latihan. Peserta tidak perlu menghafal seluruh istilah. Yang lebih penting adalah memahami hubungan antarkomponen dan mampu menjelaskan alasan di balik sebuah pilihan.

Capaian Pembelajaran

Setelah menyelesaikan modul, peserta mampu:

- menjelaskan fondasi dan istilah utama Back-end Development for AI Products;
- menerjemahkan kebutuhan menjadi rancangan yang dapat diuji;
- membuat prototipe kecil dengan dokumentasi dan kontrol dasar;
- mengevaluasi kualitas, risiko, keterbatasan, serta biaya;
- menyampaikan hasil dan rekomendasi kepada pemangku kepentingan.

Prasyarat

- Python, database dasar, dan HTTP dasar.
- Mampu membaca diagram sederhana dan mengikuti contoh kode.
- Bersedia mencatat asumsi, kegagalan, dan keputusan selama latihan.

Cara Belajar yang Disarankan

1. Baca gambaran dan tujuan setiap bab.
2. Buat catatan istilah dengan bahasa sendiri.
3. Kerjakan checkpoint sebelum melihat kembali materi.
4. Jalankan atau adaptasi contoh kode bila relevan.
5. Gunakan mini project sebagai portofolio, bukan sekadar tugas akhir.
6. Lakukan review keamanan, privasi, dan dampak sebelum demonstrasi.

Peta Materi

Tahap	Fokus	Hasil
Fondasi	Istilah, tujuan, konteks, dan arsitektur	Peta masalah yang jelas
Pembangunan	Data, proses, komponen, dan integrasi	Baseline yang dapat diuji
Evaluasi	Kualitas, risiko, subgroup, biaya	Bukti dan daftar keterbatasan
Operasi	Monitoring, ownership, respons, iterasi	Sistem yang dapat dipelihara

Bab 1 - Arsitektur Back-end dan Domain

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran arsitektur back-end dan domain dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Arsitektur Back-end dan Domain tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
layer	konsep penting dalam arsitektur back-end dan domain yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana layer diukur, diuji, atau dikendalikan?
domain	konsep penting dalam arsitektur back-end dan domain yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana domain diukur, diuji, atau dikendalikan?
boundary	konsep penting dalam arsitektur back-end dan domain yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana boundary diukur, diuji, atau dikendalikan?
dependency	konsep penting dalam arsitektur back-end dan domain yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana dependency diukur, diuji, atau dikendalikan?
service	konsep penting dalam arsitektur back-end dan domain yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana service diukur, diuji, atau dikendalikan?
repository	konsep penting dalam arsitektur back-end dan domain yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana repository diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **layer** -> **domain** -> **boundary** -> **dependency**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku. Contohnya, perubahan pada **layer** dapat memengaruhi **domain**, lalu mengubah cara **boundary** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan arsitektur back-end dan domain dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk layer serta domain.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan arsitektur back-end dan domain secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **layer** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan arsitektur back-end dan domain.
- Menganggap layer sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan layer dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara domain dan boundary dalam bab ini?
3. Sebutkan satu risiko ketika arsitektur back-end dan domain diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan layer, domain, boundary, dependency. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 2 - HTTP, REST, dan API Design

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran http, rest, dan api design dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

HTTP, REST, dan API Design tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
method	konsep penting dalam http, rest, dan api design yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana method diukur, diuji, atau dikendalikan?
resource	konsep penting dalam http, rest, dan api design yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana resource diukur, diuji, atau dikendalikan?
status code	konsep penting dalam http, rest, dan api design yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana status code diukur, diuji, atau dikendalikan?
pagination	konsep penting dalam http, rest, dan api design yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana pagination diukur, diuji, atau dikendalikan?
versioning	konsep penting dalam http, rest, dan api design yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana versioning diukur, diuji, atau dikendalikan?
OpenAPI	konsep penting dalam http, rest, dan api design yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana OpenAPI diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **method** -> **resource** -> **status code** -> **pagination**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **method** dapat memengaruhi **resource**, lalu mengubah cara **status code** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan http, rest, dan api design dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk method serta resource.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan http, rest, dan api design secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **method** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan http, rest, dan api design.
- Menganggap method sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan method dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara resource dan status code dalam bab ini?
3. Sebutkan satu risiko ketika http, rest, dan api design diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan method, resource, status code, pagination. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 3 - Data Modeling dan Database

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran data modeling dan database dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Data Modeling dan Database tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
entity	konsep penting dalam data modeling dan database yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana entity diukur, diuji, atau dikendalikan?
relation	konsep penting dalam data modeling dan database yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana relation diukur, diuji, atau dikendalikan?
index	konsep penting dalam data modeling dan database yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana index diukur, diuji, atau dikendalikan?
transaction	konsep penting dalam data modeling dan database yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana transaction diukur, diuji, atau dikendalikan?
migration	konsep penting dalam data modeling dan database yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana migration diukur, diuji, atau dikendalikan?
consistency	konsep penting dalam data modeling dan database yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana consistency diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **entity** -> **relation** -> **index** -> **transaction**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **entity** dapat memengaruhi **relation**, lalu mengubah cara **index** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan data modeling dan database dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk entity serta relation.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan data modeling dan database secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **entity** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan data modeling dan database.
- Menganggap entity sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan entity dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara relation dan index dalam bab ini?
3. Sebutkan satu risiko ketika data modeling dan database diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan entity, relation, index, transaction. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 4 - Authentication dan Authorization

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran authentication dan authorization dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Authentication dan Authorization tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
identity	konsep penting dalam authentication dan authorization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana identity diukur, diuji, atau dikendalikan?
session	konsep penting dalam authentication dan authorization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana session diukur, diuji, atau dikendalikan?
token	konsep penting dalam authentication dan authorization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana token diukur, diuji, atau dikendalikan?
role	konsep penting dalam authentication dan authorization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana role diukur, diuji, atau dikendalikan?
permission	konsep penting dalam authentication dan authorization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana permission diukur, diuji, atau dikendalikan?
policy	konsep penting dalam authentication dan authorization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana policy diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **identity** -> **session** -> **token** -> **role**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **identity** dapat memengaruhi **session**, lalu mengubah cara **token** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan authentication dan authorization dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk identity serta session.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan authentication dan authorization secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **identity** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan authentication dan authorization.
- Menganggap identity sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan identity dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara session dan token dalam bab ini?
3. Sebutkan satu risiko ketika authentication dan authorization diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan identity, session, token, role. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 5 - Service Layer dan Dependency Management

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran service layer dan dependency management dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Service Layer dan Dependency Management tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
service	konsep penting dalam service layer dan dependency management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana service diukur, diuji, atau dikendalikan?
interface	konsep penting dalam service layer dan dependency management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana interface diukur, diuji, atau dikendalikan?
dependency injection	konsep penting dalam service layer dan dependency management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana dependency injection diukur, diuji, atau dikendalikan?
configuration	konsep penting dalam service layer dan dependency management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana configuration diukur, diuji, atau dikendalikan?
error	konsep penting dalam service layer dan dependency management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana error diukur, diuji, atau dikendalikan?
testability	konsep penting dalam service layer dan dependency management yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana testability diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **service** -> **interface** -> **dependency injection** -> **configuration**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **service** dapat memengaruhi **interface**, lalu mengubah cara **dependency injection** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan service layer dan dependency management dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk service serta interface.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan service layer dan dependency management secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **service** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan service layer dan dependency management.
- Menganggap service sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan service dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara interface dan dependency injection dalam bab ini?
3. Sebutkan satu risiko ketika service layer dan dependency management diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan service, interface, dependency injection, configuration. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 6 - Asynchronous Job dan Message Queue

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran asynchronous job dan message queue dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Asynchronous Job dan Message Queue tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
queue	konsep penting dalam asynchronous job dan message queue yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana queue diukur, diuji, atau dikendalikan?
worker	konsep penting dalam asynchronous job dan message queue yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana worker diukur, diuji, atau dikendalikan?
retry	konsep penting dalam asynchronous job dan message queue yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana retry diukur, diuji, atau dikendalikan?
dead letter	konsep penting dalam asynchronous job dan message queue yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana dead letter diukur, diuji, atau dikendalikan?
idempotency	sifat proses yang aman diulang tanpa menggandakan efek	Pertanyaan yang perlu dijawab: bagaimana idempotency diukur, diuji, atau dikendalikan?
schedule	konsep penting dalam asynchronous job dan message queue yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana schedule diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **queue** -> **worker** -> **retry** -> **dead letter**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **queue** dapat memengaruhi **worker**, lalu mengubah cara **retry** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan asynchronous job dan message queue dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk queue serta worker.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan asynchronous job dan message queue secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **queue** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan asynchronous job dan message queue.
- Menganggap queue sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan queue dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara worker dan retry dalam bab ini?
3. Sebutkan satu risiko ketika asynchronous job dan message queue diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan queue, worker, retry, dead letter. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 7 - Integrasi Model, RAG, dan Tool

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran integrasi model, rag, dan tool dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Integrasi Model, RAG, dan Tool tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
model client	konsep penting dalam integrasi model, rag, dan tool yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana model client diukur, diuji, atau dikendalikan?
prompt	konsep penting dalam integrasi model, rag, dan tool yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana prompt diukur, diuji, atau dikendalikan?
retrieval	proses menemukan informasi relevan dari sumber yang tersedia	Pertanyaan yang perlu dijawab: bagaimana retrieval diukur, diuji, atau dikendalikan?
tool	konsep penting dalam integrasi model, rag, dan tool yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana tool diukur, diuji, atau dikendalikan?
citation	penunjuk sumber yang memungkinkan klaim diperiksa	Pertanyaan yang perlu dijawab: bagaimana citation diukur, diuji, atau dikendalikan?
fallback	konsep penting dalam integrasi model, rag, dan tool yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana fallback diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **model client** -> **prompt** -> **retrieval** -> **tool**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku. Contohnya, perubahan pada **model client** dapat memengaruhi **prompt**, lalu mengubah cara **retrieval** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan integrasi model, rag, dan tool dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk model client serta prompt.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan integrasi model, rag, dan tool secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **model client** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan integrasi model, rag, dan tool.
- Menganggap model client sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan model client dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara prompt dan retrieval dalam bab ini?
3. Sebutkan satu risiko ketika integrasi model, rag, dan tool diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan model client, prompt, retrieval, tool. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 8 - Caching, Rate Limit, dan Performa

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran caching, rate limit, dan performa dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Caching, Rate Limit, dan Performa tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
cache	konsep penting dalam caching, rate limit, dan performa yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana cache diukur, diuji, atau dikendalikan?
TTL	konsep penting dalam caching, rate limit, dan performa yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana TTL diukur, diuji, atau dikendalikan?
invalidation	konsep penting dalam caching, rate limit, dan performa yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana invalidation diukur, diuji, atau dikendalikan?
rate limit	konsep penting dalam caching, rate limit, dan performa yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana rate limit diukur, diuji, atau dikendalikan?
pooling	konsep penting dalam caching, rate limit, dan performa yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana pooling diukur, diuji, atau dikendalikan?
profiling	konsep penting dalam caching, rate limit, dan performa yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana profiling diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **cache** -> **TTL** -> **invalidation** -> **rate limit**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **cache** dapat memengaruhi **TTL**, lalu mengubah cara **invalidation** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan caching, rate limit, dan performa dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk cache serta TTL.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan caching, rate limit, dan performa secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **cache** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan caching, rate limit, dan performa.
- Menganggap cache sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan cache dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara TTL dan invalidation dalam bab ini?
3. Sebutkan satu risiko ketika caching, rate limit, dan performa diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan cache, TTL, invalidation, rate limit. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 9 - Testing, Security, dan Observability

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran testing, security, dan observability dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Testing, Security, dan Observability tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
unit	konsep penting dalam testing, security, dan observability yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana unit diukur, diuji, atau dikendalikan?
integration	konsep penting dalam testing, security, dan observability yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana integration diukur, diuji, atau dikendalikan?
contract	konsep penting dalam testing, security, dan observability yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana contract diukur, diuji, atau dikendalikan?
audit	konsep penting dalam testing, security, dan observability yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana audit diukur, diuji, atau dikendalikan?
log	konsep penting dalam testing, security, dan observability yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana log diukur, diuji, atau dikendalikan?
trace	konsep penting dalam testing, security, dan observability yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana trace diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **unit** -> **integration** -> **contract** -> **audit**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **unit** dapat memengaruhi **integration**, lalu mengubah cara **contract** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan testing, security, dan observability dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk unit serta integration.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan testing, security, dan observability secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **unit** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan testing, security, dan observability.
- Menganggap unit sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan unit dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara integration dan contract dalam bab ini?
3. Sebutkan satu risiko ketika testing, security, dan observability diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan unit, integration, contract, audit. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 10 - Deployment, Migration, dan Operasi

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran deployment, migration, dan operasi dalam Back-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Deployment, Migration, dan Operasi tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Back-end seperti dapur dan gudang di balik sebuah restoran: pengguna tidak melihat seluruh prosesnya, tetapi konsistensi, keamanan, dan kecepatan layanan bergantung padanya.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
container	konsep penting dalam deployment, migration, dan operasi yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana container diukur, diuji, atau dikendalikan?
CI/CD	konsep penting dalam deployment, migration, dan operasi yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana CI/CD diukur, diuji, atau dikendalikan?
migration	konsep penting dalam deployment, migration, dan operasi yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana migration diukur, diuji, atau dikendalikan?
backup	konsep penting dalam deployment, migration, dan operasi yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana backup diukur, diuji, atau dikendalikan?
rollback	mengembalikan sistem ke versi stabil sebelumnya	Pertanyaan yang perlu dijawab: bagaimana rollback diukur, diuji, atau dikendalikan?
runbook	konsep penting dalam deployment, migration, dan operasi yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana runbook diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **container** -> **CI/CD** -> **migration** -> **backup**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **container** dapat memengaruhi **CI/CD**, lalu mengubah cara **migration** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan deployment, migration, dan operasi dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk container serta CI/CD.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Back-end Knowledge Assistant**, tim perlu menerapkan deployment, migration, dan operasi secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **container** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan deployment, migration, dan operasi.
- Menganggap container sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan container dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara CI/CD dan migration dalam bab ini?
3. Sebutkan satu risiko ketika deployment, migration, dan operasi diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan container, CI/CD, migration, backup. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 11 - Mini Project: Back-end Knowledge Assistant

Tujuan Proyek

Membangun API untuk mengunggah dokumen, memproses indeks secara asynchronous, melakukan pencarian, dan menghasilkan jawaban dengan kontrol akses.

Proyek harus cukup kecil untuk selesai, tetapi cukup lengkap untuk memperlihatkan alur berpikir. Nilai utama bukan pada tampilan atau kompleksitas, melainkan kejelasan tujuan, kualitas keputusan, pengujian, dan dokumentasi.

Deliverable

- problem statement dan intended use;
- diagram arsitektur atau alur kerja;
- data dictionary atau inventaris sumber;
- baseline yang dapat dijalankan;
- hasil pengujian normal, error, dan kasus batas;
- daftar risiko serta kontrol;
- ringkasan hasil untuk audiens nonteknis;
- README dan rencana pengembangan.

Tahapan Pengerjaan

1. Tulis masalah, pengguna, keputusan, dan batas penggunaan.
2. Buat inventaris data, sumber daya, pemilik, izin, serta risiko.
3. Rancang arsitektur atau alur kerja versi minimum.
4. Bangun baseline yang dapat dijalankan ulang.
5. Tambahkan validasi, logging, dan penanganan error.
6. Susun golden set atau skenario uji, termasuk kasus batas.
7. Ukur outcome, guardrail, performa, dan biaya.
8. Dokumentasikan hasil, keterbatasan, runbook, serta rencana iterasi.

Contoh Kode Awal

```
from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()

def require_role(role: str):
    def checker(user = Depends(current_user)):
        if role not in user.roles:
            raise HTTPException(status_code=403, detail="Akses ditolak")
        return user
    return checker

@app.post("/documents")
def create_document(payload: DocumentIn, user=Depends(require_role("editor"))):
    document = service.create(payload, owner_id=user.id)
    queue.enqueue("index_document", document.id)
    return {"id": document.id, "status": "queued"}
```

Kode tersebut hanya titik awal. Tambahkan pemeriksaan input, konfigurasi, test, logging, serta penanganan error sesuai konteks proyek.

Rubrik Penilaian

Aspek	Bobot	Indikator
Problem framing	15%	Pengguna, keputusan, outcome, dan batas jelas
Data atau input	15%	Sumber, izin, kualitas, dan representasi dijelaskan
Rancangan	15%	Komponen, interface, serta trade-off

Aspek	Bobot	Indikator
		masuk akal
Implementasi	20%	Baseline berjalan, modular, dan dapat diulang
Evaluasi	15%	Metrik, kasus batas, serta subgroup diperiksa
Responsible practice	10%	Privasi, keamanan, fairness, dan human review
Komunikasi	10%	Dokumentasi dan demo mudah dipahami

Pertanyaan Review Proyek

1. Keputusan apa yang benar-benar dibantu proyek ini?
2. Apa kegagalan yang paling mungkin dan paling berbahaya?
3. Bukti apa yang menunjukkan solusi lebih baik daripada baseline?
4. Siapa yang berwenang menyetujui, menghentikan, atau mengubah sistem?
5. Informasi apa yang harus terlihat oleh pengguna agar tidak salah memahami hasil?

Bab 12 - Kuis Akhir

Pilih jawaban yang paling tepat.

1. Ketika memulai **Arsitektur Back-end dan Domain**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji layer
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
2. Ketika memulai **HTTP, REST, dan API Design**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji method
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
3. Ketika memulai **Data Modeling dan Database**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji entity
 - D. Mengukur keberhasilan hanya dari jumlah fitur
4. Ketika memulai **Authentication dan Authorization**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji identity
5. Ketika memulai **Service Layer dan Dependency Management**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji service
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
6. Ketika memulai **Asynchronous Job dan Message Queue**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji queue
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
7. Ketika memulai **Integrasi Model, RAG, dan Tool**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji model client
 - D. Mengukur keberhasilan hanya dari jumlah fitur
8. Ketika memulai **Caching, Rate Limit, dan Performa**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji cache
9. Ketika memulai **Testing, Security, dan Observability**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji unit
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
10. Ketika memulai **Deployment, Migration, dan Operasi**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji container
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur

11. Input penting berubah tanpa pemberitahuan. Kontrol terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Kontrak, validasi, versioning, dan alert
 - D. menghapus logging untuk menghemat ruang
12. Hasil sistem terlihat baik pada data latihan tetapi buruk pada konteks baru. Langkah terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Memeriksa split, leakage, representasi data, dan generalisasi
13. Tim tidak tahu siapa yang harus merespons alarm. Perbaikan utama adalah ...
- A. Menetapkan ownership, severity, runbook, dan escalation
 - B. menambah kompleksitas tanpa diagnosis
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
14. Perubahan besar akan diluncurkan. Cara menurunkan blast radius adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. Rilis bertahap, observasi, dan rollback
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
15. Pengguna tidak memahami keterbatasan hasil. Desain yang tepat adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Menjelaskan sumber, ketidakpastian, batas penggunaan, dan koreksi
 - D. menghapus logging untuk menghemat ruang
16. Data sensitif digunakan untuk latihan. Prinsip pertama adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Minimization, izin, kontrol akses, dan penggunaan data sintetis bila memungkinkan
17. Satu metrik meningkat tetapi dampak keseluruhan memburuk. Tim perlu ...
- A. Menggunakan outcome metric dan guardrail secara bersamaan
 - B. menambah kompleksitas tanpa diagnosis
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
18. Sistem gagal pada sebagian kelompok. Respons yang benar adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. Menganalisis subgroup, dampak, akar masalah, dan mitigasi
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
19. Biaya naik tanpa kenaikan nilai. Langkah awal adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Mengukur penggunaan per komponen dan menghubungkannya dengan outcome
 - D. menghapus logging untuk menghemat ruang
20. Temuan belum cukup kuat untuk keputusan otomatis. Pilihan aman adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Membatasi penggunaan sebagai dukungan keputusan dan menambahkan human review

Kunci dan Pembahasan

No.	Jawaban	Pembahasan
1	A	Pendekatan dimulai dari tujuan dan

No.	Jawaban	Pembahasan
		definisi operasional layer, kemudian alat dipilih sesuai kebutuhan.
2	B	Pendekatan dimulai dari tujuan dan definisi operasional method, kemudian alat dipilih sesuai kebutuhan.
3	C	Pendekatan dimulai dari tujuan dan definisi operasional entity, kemudian alat dipilih sesuai kebutuhan.
4	D	Pendekatan dimulai dari tujuan dan definisi operasional identity, kemudian alat dipilih sesuai kebutuhan.
5	A	Pendekatan dimulai dari tujuan dan definisi operasional service, kemudian alat dipilih sesuai kebutuhan.
6	B	Pendekatan dimulai dari tujuan dan definisi operasional queue, kemudian alat dipilih sesuai kebutuhan.
7	C	Pendekatan dimulai dari tujuan dan definisi operasional model client, kemudian alat dipilih sesuai kebutuhan.
8	D	Pendekatan dimulai dari tujuan dan definisi operasional cache, kemudian alat dipilih sesuai kebutuhan.
9	A	Pendekatan dimulai dari tujuan dan definisi operasional unit, kemudian alat dipilih sesuai kebutuhan.
10	B	Pendekatan dimulai dari tujuan dan definisi operasional container, kemudian alat dipilih sesuai kebutuhan.
11	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
12	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
13	A	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
14	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
15	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
16	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
17	A	Pilihan tersebut membuat masalah

No.	Jawaban	Pembahasan
		dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
18	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
19	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
20	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.

Interpretasi Skor

- **17-20 benar:** siap mengerjakan proyek dengan pengawasan ringan.
- **13-16 benar:** pemahaman baik, ulangi bagian yang masih lemah.
- **9-12 benar:** pelajari kembali konsep inti dan latihan.
- **0-8 benar:** ulangi modul secara bertahap dengan mentor atau teman belajar.

Bab 13 - Diskusi dan Refleksi

Pertanyaan Diskusi

1. Bagian mana dari Back-end Development for AI Products yang paling mudah diremehkan tetapi berpotensi menimbulkan kegagalan besar?
2. Kapan solusi sederhana lebih baik daripada solusi yang lebih otomatis atau kompleks?
3. Metrik apa yang dapat mendorong perilaku salah bila digunakan sendirian?
4. Bagaimana pengguna dapat mengoreksi sistem dan melihat tindak lanjutnya?
5. Siapa yang menerima manfaat dan siapa yang mungkin menerima risiko?

Jurnal Refleksi

Tuliskan satu halaman yang menjawab:

- konsep yang paling mengubah cara berpikir;
- asumsi yang sebelumnya dianggap benar;
- kesalahan yang terjadi ketika latihan;
- bukti yang masih kurang;
- satu tindakan belajar berikutnya.

Bab 14 - Glosarium dan Checklist

Glosarium

Istilah	Makna ringkas
layer	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
domain	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
boundary	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
dependency	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
service	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
repository	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
method	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
resource	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
status code	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
pagination	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
versioning	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
OpenAPI	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
entity	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
relation	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
index	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
transaction	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.

Istilah	Makna ringkas
	dipakai.
migration	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
consistency	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
identity	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
session	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
token	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
role	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
permission	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
policy	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
interface	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
dependency injection	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
configuration	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
error	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
testability	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
queue	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
worker	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
retry	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
dead letter	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum

Istilah	Makna ringkas
	dipakai.
idempotency	sifat proses yang aman diulang tanpa menggandakan efek.
schedule	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
model client	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
prompt	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
retrieval	proses menemukan informasi relevan dari sumber yang tersedia.
tool	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
citation	penunjuk sumber yang memungkinkan klaim diperiksa.
fallback	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
cache	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
TTL	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
invalidation	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
rate limit	konsep penting dalam back-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.

Checklist Sebelum Implementasi

- [] Masalah, pengguna, dan keputusan telah dinyatakan.
- [] Intended use dan penggunaan yang dilarang telah ditulis.
- [] Sumber data atau input memiliki izin dan provenance.
- [] Baseline dan kriteria penerimaan tersedia.
- [] Kondisi error, data kosong, dan kasus ekstrem diuji.
- [] Privasi, keamanan, bias, dan aksesibilitas ditinjau.
- [] Log, metric, owner, alert, dan runbook ditetapkan.
- [] Perubahan memiliki versioning, approval, dan rollback.
- [] Hasil dikomunikasikan bersama keterbatasannya.
- [] Jadwal review dan penghentian sistem ditentukan.

Bab 15 - Ringkasan dan Referensi

Ringkasan Cepat

1. Mulai dari keputusan dan pengguna, bukan alat.
2. Definisikan istilah, data, outcome, dan guardrail secara operasional.

3. Bangun baseline kecil yang dapat diamati dan diuji ulang.
4. Perlakukan keamanan, privasi, aksesibilitas, serta human review sebagai bagian desain.
5. Uji kondisi normal, kegagalan, subgroup, perubahan konteks, dan biaya.
6. Dokumentasikan provenance, asumsi, versi, owner, serta keterbatasan.
7. Gunakan monitoring dan feedback untuk memutuskan iterasi, rollback, atau penghentian.

Referensi Utama

- Designing Data-Intensive Applications - Martin Kleppmann.
- Domain-Driven Design - Eric Evans.
- OWASP Application Security Verification Standard.
- FastAPI Documentation.
- PostgreSQL Documentation.

Penutup

Kemampuan dalam Back-end Development for AI Products tidak hanya ditunjukkan oleh banyaknya alat yang dikuasai. Kemampuan terlihat dari cara peserta merumuskan masalah, memilih pendekatan yang proporsional, menguji klaim, melindungi pengguna, dan menjaga sistem tetap dapat dipertanggungjawabkan.

HerAI Fellowship - Participant Module Versi 1.0.0 - 23 Juli 2026